Intoduction and Explanation

AI Code Review Assistant

Internship Project Documentation (Python Tech Stack)

Introduction

The **AI Code Review Assistant** is an AI-powered full-stack web application that helps developers and students improve the quality of their code through automated analysis. Instead of relying solely on manual code reviews, users can upload source code files, paste code snippets, or provide a public GitHub repository URL. The application then analyzes the code using Python-based static analysis tools and an AI model to identify bugs, security vulnerabilities, code smells, performance issues, and opportunities for improvement.

This project introduces students to modern software engineering practices, including AI integration, REST APIs, authentication, database management, GitHub API integration, and automated code analysis. It closely resembles real-world developer productivity tools, making it an excellent internship-level assignment.

Project Explanation

The AI Code Review Assistant automates the process of reviewing source code using a combination of **Python static analysis tools** and **Large Language Models (LLMs).**

A user signs into the application and can submit code in the following ways:

- Upload one or more source code files
- Paste code snippets directly into the online code editor

Once the code is submitted, the backend extracts the source code and performs a multi-stage analysis.

Stage 1: File Processing

The application reads the uploaded source code files while ignoring unsupported file types such as images, binaries, and dependency folders.

Stage 2: Static Code Analysis

The backend analyzes the uploaded code using:

- **Pylint** for code quality and coding standards
- **Bandit** for security vulnerability detection
- **Radon** for complexity analysis

These tools generate reports about:

- Syntax errors
- Unused variables
- Code quality issues
- Security vulnerabilities
- Maintainability score
- Cyclomatic complexity

Stage 3: AI-Powered Review

The processed code is sent to an AI model, which generates:

- Bug detection
- Code smell analysis
- Performance improvements
- Security recommendations
- Refactoring suggestions
- Function explanations
- Documentation generation

The results are displayed in a structured dashboard, and every review is saved for future reference.

Project Objectives

- Build a production-ready Python web application
- Learn Flask backend development
- Integrate AI services into web applications

- Understand static code analysis
- Implement JWT authentication
- Design relational databases
- Develop REST APIs
- Build secure file upload functionality
- Improve frontend-backend communication
- Deploy a complete web application

Core Features

Core Features

User Authentication

- Register
- Login
- Logout
- Reset Password
- Update Profile

Code Submission

The application should support:

- Uploading Python files
- Uploading JavaScript files *(optional)*
- Pasting code snippets into an editor

Static Analysis

The backend automatically performs:

- Pylint Analysis
- Bandit Security Scan
- Radon Complexity Analysis

The reports should include:

- Code Quality Score
- Security Issues
- Complexity Metrics
- Maintainability Index
- Code Smells

AI Code Review

The AI should generate:

- Bug Reports
- Optimization Suggestions
- Performance Recommendations
- Better Naming Suggestions
- Refactoring Advice
- Best Practices
- Security Improvements

Complexity Analysis

The dashboard should display:

- Cyclomatic Complexity
- Maintainability Index
- Number of Classes
- Number of Functions
- Total Lines of Code
- Average Function Length

Documentation Generator

Automatically generate:

- Function Documentation
- Class Documentation
- Module Documentation
- README Summary *(Optional)*

Review Dashboard

Users should be able to:

- View Previous Reviews

- Search Reviews
- Filter Reviews
- Delete Reviews
- View Detailed Reports

Export Reports *(Optional)*

Allow users to export reports as:

- PDF
- Markdown
- HTML

Tech Stack

Suggested Tech Stack

Layer	Technology
Frontend	React.js
Styling	Tailwind CSS
Backend	Flask/FastApi
API Framework	Flask Blueprints / Flask-RESTful
ORM	SQLAlchemy
Database	PostgreSQL/Supabase
Authentication	Flask-JWT-Extended/Supabase
Password Hashing	Bcrypt
AI Integration	OpenAI API or Any LLM Provider
Static Analysis	Pylint, Bandit, Radon
File Upload	Flask + Werkzeug
PDF Reports	ReportLab
Deployment	Render / Railway
Frontend Hosting	Vercel

Suggested Project Structure

```
backend/
│
├── app.py
├── config.py
├── requirements.txt
│
├── models/
│   ├── user.py
│   ├── project.py
│   └── review.py
│
├── routes/
│   ├── auth.py
│   ├── upload.py
│   ├── review.py
│   └── report.py
│
├── services/
│   ├── openai_service.py
│   ├── pylint_service.py
│   ├── bandit_service.py
│   ├── radon_service.py
│   └── documentation_service.py
│
├── uploads/
├── reports/
├── utils/
└── static/

frontend/
│
├── components/
├── pages/
├── services/
├── hooks/
└── assets/
```

Database Design

Users

- id
- name
- email
- password_hash
- created_at

Projects

- id
- user_id
- project_name
- upload_type
- created_at

Reviews

- id
- project_id
- review_score
- summary
- created_at

Review Findings

- id
- review_id
- severity
- issue
- explanation
- suggestion
- file_name
- line_number

Two-Week Development Schedule

Two-Week Development Schedule

Day	Tasks
Day 1	Requirement analysis, UI wireframes, Flask project setup
Day 2	Database design and JWT authentication
Day 3	Dashboard UI and routing
Day 4	File upload API
Day 5	Integrate Pylint for code quality analysis
Day 6	Integrate Bandit and Radon for security and complexity analysis
Day 7	Display static analysis reports
Day 8	Integrate OpenAI API for AI-powered code review
Day 9	Display AI review results with severity levels
Day 10	Generate documentation automatically
Day 11	Review history, search, and filtering
Day 12	Testing, validation, and exception handling
Day 13	UI improvements, optimization, and code refactoring
Day 14	Deployment, documentation, and project presentation

Learning Outcomes

Learning Outcomes

By completing this project, students will learn:

- Python backend development using Flask
- REST API development
- SQLAlchemy ORM
- PostgreSQL database design
- JWT authentication
- GitHub REST API integration
- File upload handling
- Static code analysis using Pylint, Bandit, and Radon
- AI API integration
- Clean software architecture
- Error handling and debugging
- Deployment of full-stack applications

Bonus Features (For Advanced Students)

Students who finish early can implement:

- GitHub OAuth Login
- Multi-language support (Python, JavaScript, Java, C++)
- Repository comparison
- AI-powered code refactoring
- Interactive code editor using Monaco Editor
- Code quality score (0–100)
- Repository analytics dashboard
- Team workspaces
- Dark and Light themes
- Email notifications after analysis
- Docker support
- CI/CD integration using GitHub Actions

Expected Deliverables

Each student should submit:

- Complete Source Code (Frontend + Backend)
- GitHub Repository
- Database Schema
- API Documentation
- README File
- Deployment Link
- Sample Test Cases
- Demo Video (5–10 minutes)

Conclusion

The **AI Code Review Assistant** built with the **Python technology stack** is a practical, industry-oriented project that combines artificial intelligence with software engineering best practices. By integrating Flask, PostgreSQL, GitHub APIs, static analysis tools such as Pylint, Bandit, and Radon, and AI-powered code review, students gain experience working on a real-world developer tool.

This project goes far beyond a basic CRUD application. It requires students to design scalable APIs, integrate multiple third-party services, manage authentication, analyze source code, and present meaningful insights through an intuitive dashboard. Completing this project equips students with the technical skills and confidence expected from interns and junior full-stack developers while providing them with a strong portfolio project that demonstrates proficiency in modern Python web development.

Resources

1. Python

Official Documentation

https://docs.python.org/3/

YouTube

- Python Full Course - freeCodeCamp
  https://youtu.be/rfscVS0vtbw
- Python Tutorial for Beginners - Programming with Mosh
  https://youtu.be/_uQrJ0TkZlc

Topics to Learn

- Functions
- OOP
- File Handling
- Exception Handling
- Modules
- Virtual Environment
- Pip

---

2. Flask

Official Documentation

https://flask.palletsprojects.com/

YouTube

- Flask Full Course - freeCodeCamp
  https://youtu.be/Z1RJmh_OqeA
- Flask Crash Course - Traversy Media
  https://youtu.be/Z1RJmh_OqeA

Topics

- Routing
- Blueprints
- Request & Response
- JSON APIs
- Error Handling

---

3. React.js

Official Documentation

https://react.dev/

YouTube

- React Full Course - freeCodeCamp
  https://youtu.be/bMknfKXIFA8
- React JS Full Course - Codevolution
  https://www.youtube.com/playlist?list=PLC3y8-rFHvwhiQJD1di4eRVN30WWCXkg1

Topics

- Components
- State
- Props
- Hooks
- Forms
- API Calls

---

4. Tailwind CSS

Official Documentation

https://tailwindcss.com/docs

YouTube

- Tailwind CSS Full Course
  https://youtu.be/lCxcTsOHrjo

Topics

- Responsive Design
- Grid
- Flexbox
- Cards
- Forms

---

5. PostgreSQL

Official Documentation

https://www.postgresql.org/docs/

YouTube

- PostgreSQL Full Course
  https://youtu.be/SpfIwlAYaKk

---

6. SQLAlchemy

Official Documentation

https://docs.sqlalchemy.org/

YouTube

- SQLAlchemy Tutorial
  https://youtu.be/AKQ3XEDI9Mw

---

7. JWT Authentication

Documentation

https://flask-jwt-extended.readthedocs.io/

YouTube

- JWT Authentication with Flask
  https://youtu.be/9N6a-VLBa2I

---

8. File Uploads in Flask

Documentation

https://flask.palletsprojects.com/en/latest/patterns/fileuploads/

YouTube

- Flask File Upload Tutorial
  https://youtu.be/GeiUTkSAJPs

---

9. OpenAI API

Official Documentation

https://platform.openai.com/docs

YouTube

- OpenAI API Crash Course
  https://youtu.be/uRQH2CFvedY

Topics

- API Keys
- Chat Completions
- Prompt Engineering
- Structured JSON Responses

---

10. Pylint

Documentation

https://pylint.pycqa.org/

YouTube

- Pylint Tutorial
  https://youtu.be/9L77QExPmI0

---

11. Bandit

Documentation

https://bandit.readthedocs.io/

YouTube

https://youtu.be/6P5sQq3nM3A

---

12. Radon

Documentation

https://radon.readthedocs.io/

YouTube

https://youtu.be/NR0D5d7QJ5Q

---

13. ReportLab (PDF Reports)

Documentation

https://www.reportlab.com/documentation/

YouTube

https://youtu.be/t6Os2DjN6qw

---

14. Axios

Documentation

https://axios-http.com/

YouTube

https://youtu.be/Z8oY6A0Q3BA

---

15. React Router

Documentation

https://reactrouter.com/

YouTube

https://youtu.be/Ul3y1LXxzdU

---

16. Chart.js

Useful for displaying:

- Code Quality Score
- Complexity Graph
- Bug Distribution

Documentation

https://www.chartjs.org/

YouTube

https://youtu.be/sE08f4iuOhA

---

17. Deployment

Backend

Render

https://render.com

Tutorial

https://youtu.be/l134cBAJCuc

Frontend

Vercel

https://vercel.com

Tutorial

https://youtu.be/IsdzxYfSHN8

---

18. Git & GitHub

Documentation

https://git-scm.com/docs

YouTube

Git & GitHub Crash Course

https://youtu.be/RGOj5yH7evk

---

19. VS Code

Download

https://code.visualstudio.com/

Useful Extensions

- Python
- Pylance
- ESLint
- Prettier
- Tailwind CSS IntelliSense

- Thunder Client
- Error Lens

---

20. API Testing

Thunder Client

https://www.thunderclient.com/

OR

Postman

https://www.postman.com/

Tutorial

https://youtu.be/VywxIQ2ZXw4

Recommended Folder Structure

AI-Code-Review-Assistant/

```
frontend/
│
├── src/
├── pages/
├── components/
├── services/
├── hooks/
└── assets/

backend/
│
├── app.py
├── config.py
├── requirements.txt
├── models/
├── routes/
```

├── services/
├── uploads/
├── reports/
└── utils/

database/

README.md

Recommended Learning Order

Students should complete the topics in the following sequence:

1. Python Refresher
2. Flask Basics
3. REST APIs
4. PostgreSQL
5. SQLAlchemy
6. JWT Authentication
7. File Uploads
8. React Frontend
9. Axios
10. OpenAI API Integration
11. Pylint
12. Bandit
13. Radon
14. Report Generation
15. Charts and Dashboard
16. Deployment

AI Prompt for Code Review

Students can use the following prompt when sending code to the AI model:

You are an experienced Senior Software Engineer.

Review the uploaded code and provide:

1. Bugs found

2. Security issues
3. Code smells
4. Complexity analysis
5. Performance improvements
6. Best practices
7. Suggested refactoring
8. Better variable/function names
9. Code quality score out of 100
10. Summary of improvements

Return the response in structured JSON format.